Date.prototype.setMinutes()

📄 Título

Date.prototype.setMinutes() – El establecedor de los minutos

📝 Definición

`Date.prototype.setMinutes()` es un método de JavaScript que establece los **minutos** (0-59) de un objeto `Date` según la **hora local**, pudiendo también establecer opcionalmente los segundos y milisegundos.

🧠 ¿Qué es?

Es el método estándar para **modificar los minutos** de una fecha en JavaScript.

Permite cambiar los minutos manteniendo el resto de componentes (año, mes, día, hora), con la capacidad de manejar **overflows** automáticamente.

El método es **mutable**: modifica el objeto `Date` original en lugar de crear uno nuevo.

🎯 ¿Para qué sirve?

- Cambiar los **minutos** de una fecha existente.
- Sumar o restar **minutos** a una fecha.
- Implementar **temporizadores** y **cronómetros**.
- Ajustar horas para **conversiones de zona horaria**.
- Establecer **minutos exactos** en fechas.
- Manejar **ajustes automáticos** de horas/días.
- Implementar cálculos de **intervalos de tiempo** en minutos.
- Crear **logs y marcas de tiempo** con precisión.

⚙️ ¿Cómo funciona?

El método establece los minutos (0-59), y opcionalmente los segundos y milisegundos, basado en la **hora local**.

Comportamiento:

- Los minutos deben ser un número entre 0 y 59.
- Si solo se pasan minutos, se mantienen los segundos y milisegundos existentes.
- Los valores fuera de rango se **ajustan automáticamente**:
  - Minutos > 59 → se ajusta a la hora siguiente.
  - Minutos < 0 → se ajusta a la hora anterior.
  - Ej: `setMinutes(90)` → 30 minutos después de la hora.
  - Ej: `setMinutes(-10)` → 50 minutos antes de la hora.
- Los segundos y milisegundos también se ajustan automáticamente si se pasan fuera de rango.

Retorna: El **timestamp** en milisegundos del objeto modificado.

📝 Sintaxis / Estructura (JavaScript)

```
date.setMinutes(minutes)
date.setMinutes(minutes, seconds)
date.setMinutes(minutes, seconds, milliseconds)
```

Parámetros:

- `minutes` (obligatorio): Número entero para los minutos (0-59, se aceptan valores fuera de rango con ajuste automático).
- `seconds` (opcional): Número entero para los segundos (0-59).
- `milliseconds` (opcional): Número entero para los milisegundos (0-999).

Retorna: El **timestamp** en milisegundos (del objeto modificado).

💡 Ejemplos

```
// 1. Uso básico - establecer solo los minutos
const date = new Date(2024, 11, 25, 10, 30);
console.log(date.getMinutes()); // 30

date.setMinutes(15);
console.log(date.getMinutes()); // 15
console.log(date.getHours()); // 10 (mantiene la hora)

// 2. Establecer minutos, segundos y milisegundos
const date2 = new Date(2024, 11, 25, 10, 30, 45, 500);
date2.setMinutes(15, 20, 300);
console.log(date2.getMinutes()); // 15
console.log(date2.getSeconds()); // 20
console.log(date2.getMilliseconds()); // 300

// 3. Ajuste automático - minutos fuera de rango
const date3 = new Date(2024, 11, 25, 10, 30);
date3.setMinutes(90);
console.log(date3.getMinutes()); // 30
console.log(date3.getHours()); // 11 (hora + 1)

// 4. Ajuste automático - minutos negativos
const date4 = new Date(2024, 11, 25, 10, 30);
date4.setMinutes(-10);
console.log(date4.getMinutes()); // 50
console.log(date4.getHours()); // 9 (hora - 1)

// 5. Sumar minutos a una fecha
function addMinutes(date, minutes) {
  const newDate = new Date(date);
  newDate.setMinutes(newDate.getMinutes() + minutes);
  return newDate;
}

const start = new Date(2024, 11, 25, 10, 30);
console.log(addMinutes(start, 45)); // 10:30 + 45 = 11:15
console.log(addMinutes(start, 90)); // 10:30 + 90 = 12:00
console.log(addMinutes(start, -30)); // 10:30 - 30 = 10:00
```

```
// 6. Ajuste con segundos y milisegundos
const date5 = new Date(2024, 11, 25, 10, 30, 45, 500);
date5.setMinutes(15, 70, 1500);
console.log(date5.getMinutes()); // 16 (15 + 1 por segundo
s)
console.log(date5.getSeconds()); // 10 (70 - 60 = 10)
console.log(date5.getMilliseconds()); // 500 (1500 - 1000 =
500)

// 7. Establecer minutos exactos en timestamp
function setExactMinutes(date, minutes) {
  const newDate = new Date(date);
  newDate.setMinutes(minutes, 0, 0);
  return newDate;
}

const d = new Date(2024, 11, 25, 10, 30, 45, 500);
console.log(setExactMinutes(d, 0)); // 10:00:00.000

// 8. Redondear a la hora exacta
function roundToHour(date) {
  const newDate = new Date(date);
  newDate.setMinutes(0, 0, 0);
  return newDate;
}

console.log(roundToHour(new Date(2024, 11, 25, 10, 30, 45,
500))); // 10:00:00.000

// 9. Redondear a los 15 minutos
function roundToNearest15Minutes(date) {
  const newDate = new Date(date);
  const minutes = newDate.getMinutes();
  const rounded = Math.round(minutes / 15) * 15;
  newDate.setMinutes(rounded, 0, 0);
  return newDate;
}
```

```
console.log(roundToNearest15Minutes(new Date(2024, 11, 25,
10, 12))); // 10:15
console.log(roundToNearest15Minutes(new Date(2024, 11, 25,
10, 18))); // 10:15
console.log(roundToNearest15Minutes(new Date(2024, 11, 25,
10, 23))); // 10:30

// 10. Cadena de métodos (retorna timestamp)
const date6 = new Date();
const timestamp = date6.setMinutes(30);
console.log(timestamp); // Número (timestamp en ms)
console.log(date6.getMinutes()); // 30

// 11. Mantener segundos y milisegundos existentes
const date7 = new Date(2024, 11, 25, 10, 30, 45, 500);
date7.setMinutes(15);
console.log(date7.getMinutes()); // 15
console.log(date7.getSeconds()); // 45 (mantiene)
console.log(date7.getMilliseconds()); // 500 (mantiene)

// 12. Comparación con otras operaciones
function isExactMinute(date, minute) {
  return date.getMinutes() === minute;
}

console.log(isExactMinute(new Date(2024, 11, 25, 10, 30), 3
0)); // true

// 13. Generar fechas con minutos específicos
function generateDatesWithMinute(year, month, day, minute)
{
  const dates = [];
  for (let hour = 0; hour < 24; hour++) {
    const date = new Date(year, month, day, hour, minute);
    dates.push(date);
  }
  return dates;
```

```
}

console.log(generateDatesWithMinute(2024, 11, 25, 30)); //
Cada hora del día con :30

// 14. Crear cronómetro simple
class SimpleTimer {
  constructor() {
    this.startTime = null;
    this.elapsed = 0;
  }

  start() {
    this.startTime = new Date();
    return this;
  }

  stop() {
    if (this.startTime) {
      this.elapsed = new Date().getTime() - this.startTime.
getTime();
    }
    return this;
  }

  getMinutes() {
    return Math.floor(this.elapsed / 60000);
  }

  getSeconds() {
    return Math.floor((this.elapsed % 60000) / 1000);
  }

  reset() {
    this.startTime = null;
    this.elapsed = 0;
    return this;
  }
```

```
}

const timer = new SimpleTimer();
timer.start();
setTimeout(() => {
  timer.stop();
  console.log(`Minutos:${timer.getMinutes()}`);
  console.log(`Segundos:${timer.getSeconds()}`);
}, 5000);
```

⚠️ Errores comunes

- **Pasar minutos fuera de rango sin entender el ajuste**:

```
const date = new Date(2024, 11, 25, 10, 30);
date.setMinutes(90);
console.log(date.getMinutes()); // 30
console.log(date.getHours()); // 11 (¡se ajustó automáti
camente!)
```

- **Confundir el orden de parámetros**:

```
// setMinutes(minutes, seconds, milliseconds)
// ❌ NO setMinutes(seconds, minutes, milliseconds)
```

- **No considerar la zona horaria**:

```
// setMinutes() trabaja en hora local
const date = new Date('2024-12-25T10:30:00Z');
date.setMinutes(15);
// Modifica la hora local, no la UTC
```

- **Olvidar que setMinutes() modifica el objeto original**:

```
const original = new Date(2024, 11, 25, 10, 30);
const copy = original;
copy.setMinutes(15);
console.log(original.getMinutes()); // 15 (¡también modificado!)

// Para evitar: const copy = new Date(original);
```

💼 Casos de uso

1. **Sistema de citas con precisión por minuto**:

```
class AppointmentScheduler {
  constructor() {
    this.appointments = [];
  }

  createAppointment(title, date, duration = 30) {
    const start = new Date(date);
    // Redondear a minutos exactos
    start.setMinutes(start.getMinutes(), 0, 0);

    const end = new Date(start);
    end.setMinutes(end.getMinutes() + duration);

    this.appointments.push({
      title,
      start,
      end,
      duration,
      status: 'scheduled'
    });

    return this.appointments[this.appointments.length - 1];
  }
```

```
  getAppointmentsByDay(date) {
    const target = new Date(date);
    target.setHours(0, 0, 0, 0);

    return this.appointments
      .filter(a => {
        const start = new Date(a.start);
        start.setHours(0, 0, 0, 0);
        return start.getTime() === target.getTime();
      })
      .sort((a, b) => a.start - b.start);
  }

  getAvailableSlots(date, duration = 30) {
    const appointments = this.getAppointmentsByDay(dat
e);
    const slots = [];
    const start = new Date(date);
    start.setHours(9, 0, 0, 0); // 9:00 AM
    const end = new Date(date);
    end.setHours(17, 0, 0, 0); // 5:00 PM

    let current = new Date(start);
    while (current < end) {
      const slotEnd = new Date(current);
      slotEnd.setMinutes(current.getMinutes() + duratio
n);

      const isBooked = appointments.some(a =>
        a.start.getTime() === current.getTime() &&
        a.start.toDateString() === current.toDateString
()
      );

      if (!isBooked) {
        slots.push({
          start: new Date(current),
```

```
          end: slotEnd,
          time: current.toLocaleTimeString('es-ES', {
            hour: '2-digit',
            minute: '2-digit'
          })
        });
      }

      current.setMinutes(current.getMinutes() + duratio
n);
    }

    return slots;
  }

  findNextAvailable(date, duration = 30) {
    const slots = this.getAvailableSlots(date, duratio
n);
    return slots.length > 0 ? slots[0] : null;
  }
}

const scheduler = new AppointmentScheduler();
scheduler.createAppointment('Reunión A', new Date(2024,
6, 15, 10, 0), 30);
scheduler.createAppointment('Reunión B', new Date(2024,
6, 15, 10, 30), 30);
console.log(scheduler.getAvailableSlots(new Date(2024,
6, 15)));
```

2. **Sistema de logs y auditoría con precisión temporal**:

```
class TimestampManager {
  constructor() {
    this.logs = [];
  }
```

```
  logEvent(event, data = null) {
    const now = new Date();
    // Normalizar milisegundos para logs
    const timestamp = new Date(now);
    timestamp.setMilliseconds(0);

    this.logs.push({
      timestamp,
      event,
      data,
      formatted: timestamp.toLocaleTimeString('es-ES', {
        hour: '2-digit',
        minute: '2-digit',
        second: '2-digit'
      })
    });
  }

  getLogsByMinute(minute) {
    return this.logs.filter(l => l.timestamp.getMinutes
() === minute);
  }

  getLogsByHour(hour) {
    return this.logs.filter(l => l.timestamp.getHours()
=== hour);
  }

  getLogsInRange(startMinute, endMinute) {
    return this.logs.filter(l => {
      const minutes = l.timestamp.getHours() * 60 + l.ti
mestamp.getMinutes();
      return minutes >= startMinute && minutes <= endMin
ute;
    });
  }

  getLogStats() {
```

```
    const stats = {};
    this.logs.forEach(log => {
      const minute = log.timestamp.getMinutes();
      stats[minute] = (stats[minute] || 0) + 1;
    });
    return stats;
  }
}

const logger = new TimestampManager();
logger.logEvent('login', { user: 'admin' });
logger.logEvent('page_view', { page: '/dashboard' });
logger.logEvent('logout', { user: 'admin' });
console.log(logger.getLogStats());
```

3. **Sistema de temporizador y recordatorios**:

```
class TimerManager {
  constructor() {
    this.timers = [];
    this.interval = null;
  }

  setTimer(name, minutes) {
    const start = new Date();
    const end = new Date(start);
    end.setMinutes(end.getMinutes() + minutes);

    this.timers.push({
      name,
      start,
      end,
      minutes,
      remaining: minutes * 60,
      active: true
    });

    this.startChecking();
```

```
    return this.timers[this.timers.length - 1];
  }

  startChecking() {
    if (this.interval) return;

    this.interval = setInterval(() => {
      const now = new Date();
      this.timers.forEach(timer => {
        if (!timer.active) return;

        const elapsed = Math.floor((now - timer.start) /
1000);
        timer.remaining = Math.max(0, timer.minutes * 60
- elapsed);

        if (timer.remaining === 0) {
          this.triggerTimer(timer);
        }
      });
    }, 1000);
  }

  triggerTimer(timer) {
    timer.active = false;
    console.log(`⏰ ¡${timer.name} ha terminado!`);
    // Aquí se podría mostrar una notificación
  }

  getTimers() {
    return this.timers.map(t => ({
      ...t,
      remainingFormatted: this.formatTime(t.remaining)
    }));
  }

  formatTime(seconds) {
    const minutes = Math.floor(seconds / 60);
```

```
    const remainingSeconds = seconds % 60;
    return `${minutes.toString().padStart(2, '0')}:${rem
ainingSeconds.toString().padStart(2, '0')}`;
  }

  stop() {
    if (this.interval) {
      clearInterval(this.interval);
      this.interval = null;
    }
  }
}

const timerManager = new TimerManager();
timerManager.setTimer('Cocinar', 5);
timerManager.setTimer('Descanso', 10);
console.log(timerManager.getTimers());
```

4. **Sistema de métricas en tiempo real**:

```
class MetricsCollector {
  constructor() {
    this.metrics = [];
  }

  recordMetric(name, value) {
    const now = new Date();
    // Normalizar a minutos exactos para agregación
    const timestamp = new Date(now);
    timestamp.setSeconds(0, 0);

    this.metrics.push({
      name,
      value,
      timestamp,
      minute: timestamp.getMinutes(),
      hour: timestamp.getHours()
```

```
    });
  }

  getMetricsByMinute(minute, name = null) {
    let filtered = this.metrics.filter(m => m.minute ===
minute);
    if (name) {
      filtered = filtered.filter(m => m.name === name);
    }
    return filtered;
  }

  getAverageByMinute(minute, name) {
    const metrics = this.getMetricsByMinute(minute, nam
e);
    if (metrics.length === 0) return 0;
    return metrics.reduce((sum, m) => sum + m.value, 0)
/ metrics.length;
  }

  getMetricsByHour(hour) {
    return this.metrics.filter(m => m.hour === hour);
  }

  getMetricsByTimeRange(startMinute, endMinute) {
    return this.metrics.filter(m =>
      m.minute >= startMinute && m.minute <= endMinute
    );
  }

  clearMetrics() {
    this.metrics = [];
  }
}

const metrics = new MetricsCollector();
metrics.recordMetric('requests', 100);
metrics.recordMetric('requests', 150);
```

```
metrics.recordMetric('errors', 5);
console.log(metrics.getAverageByMinute(new Date().getMin
utes(), 'requests'));
```

5. **Sistema de programación de tareas recurrentes**:

```
class TaskScheduler {
  constructor() {
    this.tasks = [];
    this.interval = null;
  }

  scheduleTask(name, hour, minute, interval = 'daily') {
    const now = new Date();
    const nextRun = new Date(now);
    nextRun.setHours(hour, minute, 0, 0);

    // Si ya pasó la hora hoy, programar para mañana
    if (nextRun < now) {
      nextRun.setDate(nextRun.getDate() + 1);
    }

    this.tasks.push({
      name,
      hour,
      minute,
      interval,
      nextRun,
      lastRun: null,
      active: true
    });

    this.startScheduler();
    return this.tasks[this.tasks.length - 1];
  }

  startScheduler() {
    if (this.interval) return;
```

```
    this.interval = setInterval(() => {
      const now = new Date();
      this.tasks.forEach(task => {
        if (!task.active) return;

        const taskTime = new Date(now);
        taskTime.setHours(task.hour, task.minute, 0, 0);

        // Verificar si es hora de ejecutar
        if (taskTime <= now && (!task.lastRun || taskTim
e > task.lastRun)) {
          this.executeTask(task);
        }
      });
    }, 60000); // Verificar cada minuto
  }

  executeTask(task) {
    task.lastRun = new Date();
    console.log(`✅ Ejecutando tarea:${task.name} a las
${task.hour}:${task.minute}`);

    // Programar siguiente ejecución
    const next = new Date(task.lastRun);
    switch(task.interval) {
      case 'daily':
        next.setDate(next.getDate() + 1);
        break;
      case 'weekly':
        next.setDate(next.getDate() + 7);
        break;
      case 'monthly':
        next.setMonth(next.getMonth() + 1);
        break;
    }
    task.nextRun = next;
  }
```

```
  getTasksByMinute(minute) {
    return this.tasks.filter(t => t.minute === minute);
  }

  getUpcomingTasks() {
    return this.tasks
      .filter(t => t.active)
      .sort((a, b) => a.nextRun - b.nextRun)
      .slice(0, 5);
  }

  stop() {
    if (this.interval) {
      clearInterval(this.interval);
      this.interval = null;
    }
  }
}

const scheduler = new TaskScheduler();
scheduler.scheduleTask('Backup DB', 2, 0); // 2:00 AM
scheduler.scheduleTask('Reporte diario', 9, 30); // 9:30
AM
console.log(scheduler.getUpcomingTasks());
```

6. **Sistema de versionado y timestamps**:

```
class VersionManager {
  constructor() {
    this.versions = [];
  }

  createVersion(label, data) {
    const now = new Date();
    const timestamp = new Date(now);
    timestamp.setMilliseconds(0);
```

```
    const version = {
      label,
      data,
      timestamp,
      versionId: this.generateVersionId(timestamp)
    };

    this.versions.push(version);
    return version;
  }

  generateVersionId(date) {
    const year = date.getFullYear();
    const month = String(date.getMonth() + 1).padStart
(2, '0');
    const day = String(date.getDate()).padStart(2, '0');
    const hours = String(date.getHours()).padStart(2,
'0');
    const minutes = String(date.getMinutes()).padStart
(2, '0');
    const seconds = String(date.getSeconds()).padStart
(2, '0');

    return `v${year}${month}${day}_${hours}${minutes}${s
econds}`;
  }

  getVersionsByMinute(minute) {
    return this.versions.filter(v => v.timestamp.getMinu
tes() === minute);
  }

  getVersionsByHour(hour) {
    return this.versions.filter(v => v.timestamp.getHour
s() === hour);
  }
```

```
  getLatestVersion() {
    return this.versions.length > 0
      ? this.versions[this.versions.length - 1]
      : null;
  }

  getVersionHistory() {
    return this.versions.map(v => ({
      id: v.versionId,
      label: v.label,
      time: v.timestamp.toLocaleTimeString('es-ES', {
        hour: '2-digit',
        minute: '2-digit',
        second: '2-digit'
      })
    }));
  }
}

const versionManager = new VersionManager();
versionManager.createVersion('Initial commit', { conten
t: 'Hello World' });
versionManager.createVersion('Update README', { content:
'Updated readme' });
console.log(versionManager.getVersionHistory());
```

🧩 Relación con otros conceptos

- `Date.prototype.getMinutes()`: Obtiene los minutos.
- `Date.prototype.setHours()`: Establece la hora.
- `Date.prototype.setSeconds()`: Establece los segundos.
- `Date.prototype.setMilliseconds()`: Establece los milisegundos.
- `Date.prototype.getTime()`: Obtiene el timestamp.
- `Date.prototype.setUTCMinutes()`: Versión en UTC.

🧠 Analogía

Imagina que `setMinutes()` es como un **reloj digital ajustable**. Puedes girar una rueda para cambiar los minutos, y si pasas de 59, automáticamente el reloj avanza una hora. Es una forma simple de ajustar el tiempo exacto manteniendo todo sincronizado.

📚 Fuente

- MDN Web Docs – Date.prototype.setMinutes()
- ECMAScript Specification – Date.prototype.setMinutes